

DR. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY, LONERE Summer Examination – 2023 Course: B. Tech.Branch : Computer Science & Engineering Semester : VI Subject Code & Name: BTCOC601 - Compiler Design Max Marks: 60 Date:12/07/2023 Duration: 3 Hr.			
Instructions to the Students: 1. All the questions are compulsory. 2. The level of question/expected answer as per OBE or the Course Outcome (CO) on which the question is based is mentioned in () in front of the question. 3. Use of non-programmable scientific calculators is allowed. 4. Assume suitable data wherever necessary and mention it clearly.			
		(Level/CO)	Marks
Q.1	Solve Any Two of the following.		12
A)	Draw a neat diagram and explain different phases of compiler. Differentiate between Interpreter, Compiler and Hybrid compiler.	Remembering	6
B)	Define token, pattern, and lexeme with suitable examples. Find tokens, pattern and lexemes for the following C- program. Give reasonable attribute values for the tokens. <pre>int main() { int a = 10, b = 20; printf ("Sum is : %d", a+b); return (0) }</pre>	Remembering	6
C)	i) Explain regular expression and regular definition with example. ii) Write regular definitions for the following languages: 1. All strings of lowercase letters that contain the five vowels in order. 2. All strings of lowercase letters in which the letters are in ascending lexicographic order.	Analyzing	6
Q.2	Solve Any Two of the following.		12
A)	Explain the two buffer input scheme for scanning the source program in lexical analyzer? How the use of sentinels can improve its performance?	Remembering	6
B)	i) Explain the role of finite automata in lexical analyzer and differentiate between NFA and DFA. ii) Convert the following regular expression to NFA using Thompson Construction algorithm. (alb*ab)*	Applying	6
C)	Convert the following NFA to DFA using subset construction algorithm. ((01 0) 1*)*	Remembering Analyzing	6
Q.3	Solve Any Two of the following.		12

A)	i) What is left recursion and left factoring? ii) Consider the CFG $S \rightarrow SS+ SS* a$ a) Left factor this grammar. b) Eliminate the left recursion from the original grammar. c) Is the resulting grammar suitable for top-down parsing?	Remembering	6
B)	What is the difference between recursive-descent parsing and predictive parsing? Draw and explain the model of non recursive predictive parser.	Remembering Applying	6
C)	What is LL(1) grammar? Construct a LL(1) parsing table for the following grammar. Test whether following grammar is LL(1) . $S \rightarrow aAB bA \epsilon$ $A \rightarrow aAb \epsilon$ $B \rightarrow bB \epsilon$	Applying	6
Q.4	Solve Any Two of the following.		12
A)	Explain stack implementation of shift reduce parsing with their four possible actions. Consider the following grammar $S \rightarrow TL$ $T \rightarrow \text{int} \text{float}$ $L \rightarrow L, \text{id} \text{id}$ Parse the input string int id, id using shift – reduce parser.	Understanding. Applying	6
B)	i) Differentiate between LR(0) and LR(1) grammar with suitable example. ii) What is handle? Consider the following grammar and show the handle of each right sentential form for the string $(a, (a, a))$. $S \rightarrow (L)a$ $L \rightarrow L, S S$	Applying Analyzing	6
C)	Show that following grammar is LL(1) but not SLR(1). $S \rightarrow AaAb BbBa$ $A \rightarrow \epsilon$ $B \rightarrow \epsilon$	Applying	6
Q.5	Solve Any Two of the following.		
A)	For the SDD given below, give annotated parse tree for the expression 3*5+4	Applying	6

	<table><tr><th>PRODUCTION</th><th>SEMANTIC RULES</th></tr><tr><td>1) $L \rightarrow E \text{ n}$</td><td>$L.val = E.val$</td></tr><tr><td>2) $E \rightarrow E_1 + T$</td><td>$E.val = E_1.val + T.val$</td></tr><tr><td>3) $E \rightarrow T$</td><td>$E.val = T.val$</td></tr><tr><td>4) $T \rightarrow T_1 * F$</td><td>$T.val = T_1.val \times F.val$</td></tr><tr><td>5) $T \rightarrow F$</td><td>$T.val = F.val$</td></tr><tr><td>6) $F \rightarrow (E)$</td><td>$F.val = E.val$</td></tr><tr><td>7) $F \rightarrow \text{digit}$</td><td>$F.val = \text{digit.lexval}$</td></tr></table>	PRODUCTION	SEMANTIC RULES	1) $L \rightarrow E \text{ n}$	$L.val = E.val$	2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$	3) $E \rightarrow T$	$E.val = T.val$	4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$	5) $T \rightarrow F$	$T.val = F.val$	6) $F \rightarrow (E)$	$F.val = E.val$	7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$		
PRODUCTION	SEMANTIC RULES																		
1) $L \rightarrow E \text{ n}$	$L.val = E.val$																		
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$																		
3) $E \rightarrow T$	$E.val = T.val$																		
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$																		
5) $T \rightarrow F$	$T.val = F.val$																		
6) $F \rightarrow (E)$	$F.val = E.val$																		
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$																		
B)	List the issues in designing code generator. Generate code for the following three-address statements assuming all variables are stored in memory locations. $x = b * c$ $y = a + x$	Applying	6																
C)	Define basic block and flow graph. Give basic blocks and flow graph for the following sequence of three address code statements: (1) $prod := 0$ (2) $i := 1$ (3) $t_1 := 4 * i$ (4) $t_2 := a[t_1]$ <i>/*compute a[i] */</i> (5) $t_3 := 4 * i$ (6) $t_4 := b[t_3]$ <i>/*compute b[i] */</i> (7) $t_5 := t_2 * t_4$ (8) $t_6 := prod + t_5$ (9) $prod := t_6$ (10) $t_7 := i + 1$ (11) $i := t_7$ (12) if $i \leq 20$ goto (3)	Remembering Understanding	6																
	*** End ***																		

The grid and the borders of the table will be hidden before final printing.